

Attention Is All You Need

Vaswani et al. — NIPS 2017

Introducing the Transformer

Today's Journey

From sequence problems to the Transformer

1

Background

Neural networks & sequence problems

3

Transformer at a Glance

The big picture

5

Multi-Head Attention

Many perspectives, in parallel

7

Why Self-Attention?

What it gains over RNN/CNN

2

The Problem

Why RNNs and CNNs aren't enough

4

Self-Attention

The core mechanism — intuition & math

6

Full Architecture

Encoder, decoder, and all the pieces

8

Results & Impact

Performance and what came next

Part 1

Background & Motivation

A 5-minute crash course on neural networks for sequences

Neural Networks — A Quick Refresher

What you need to know going in

1 2
3 4

Vectors & Embeddings

Words become numbers.
Each word → vector of (e.g. 512) numbers.
Similar words → similar vectors.



Layers & Weights

Stack of operations on vectors.
Each layer has learned weights W .
Output = some function of $(\text{input} \times W)$.



Training

Compare prediction to correct answer.
Adjust weights to reduce error.
Repeat millions of times.

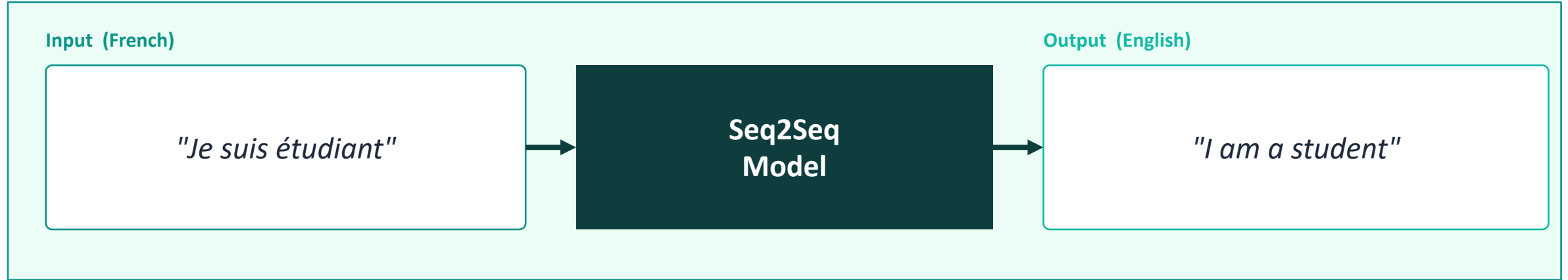


The whole game in NLP

Turn words into vectors → do clever things with those vectors → turn them back into words.
The "clever things" are what distinguish different architectures (RNN vs CNN vs Transformer).

The Sequence-to-Sequence Problem

Machine translation as the canonical example



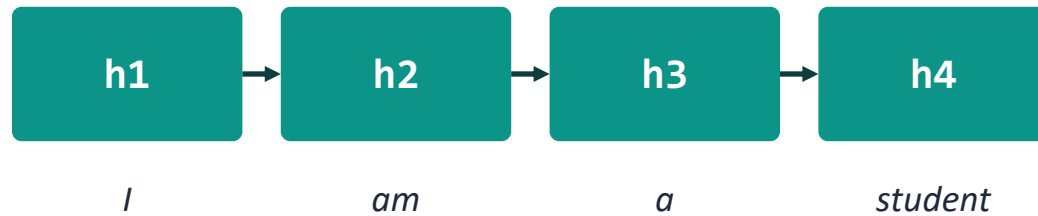
What's tricky about this?

- ! **Variable length** — Input and output sentences are different lengths
- ! **Word order matters** — Same words, different order → different meaning
- ! **Long-range dependencies** — Words far apart can refer to each other
- ! **Context is everything** — Meaning of a word depends on surrounding words

The Old Way: Recurrent Neural Networks

How they work, and where they hurt

How an RNN processes a sentence



State flows left to right, one word at a time

⚠️ Two big problems

1. Inherently sequential

Can't process h_3 until h_2 is done.

→ *slow training, no parallelization.*

2. Long-range dependencies fade

Info from word 1 has to pass through $h_1 \rightarrow h_2 \rightarrow \dots \rightarrow h_{100}$.

→ *"forgetting" over distance.*

The big question

*Can we build a sequence model that processes all positions in parallel,
AND connects any two words in a single step — no matter how far apart?*

→ **That's exactly what the Transformer does.**

Part 2

The Transformer

at a Glance

Look at the whole picture before zooming in

Opening the Black Box

Three levels of zoom

Level 1: As a black box



Level 2: Encoder + Decoder



Level 3: Stacks of 6 identical layers each



Encoder × 6



Decoder × 6

Each layer:

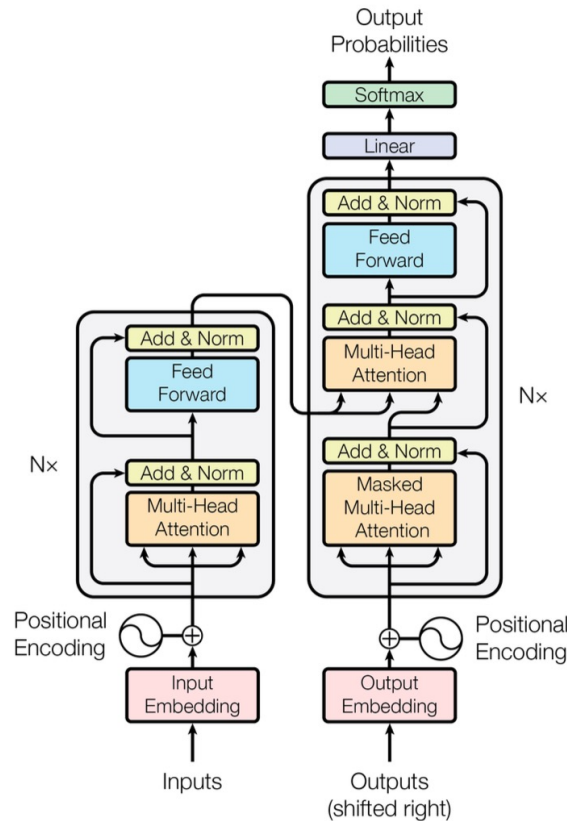
- Self-attention
- Feed-forward NN
- (*decoder: + cross-attn*)

The Full Architecture

Figure 1 from the paper — we'll unpack this piece by piece

ENCODER (left)

- Input embedding + positional encoding
- Multi-Head Self-Attention
- Feed-Forward Network
- Add & Norm (residual + layer norm)
- Repeat $\times 6$



DECODER (right)

- Output embedding + positional encoding
- Masked Multi-Head Self-Attention
- Cross-Attention (looks at encoder)
- Feed-Forward Network
- Add & Norm everywhere
- Linear + Softmax $\rightarrow$ next-word probs

Figure 1: The Transformer - model architecture.

We'll spend most of today on the orange boxes — the attention mechanisms — which are the new idea.

Step 0: Preparing the Input

From words to vectors

1. Word Embedding

Each word $\rightarrow$ vector of $d_{\text{model}} = 512$ numbers

Example:

"student" $\rightarrow$ $[0.21, -0.43, 0.85, \dots, 0.07]$

2. Positional Encoding

Problem: attention treats input as a **set** — order is lost.

Fix: add a position-dependent vector to each embedding.

Paper uses **sine & cosine** of different frequencies.

Final input = embedding + positional encoding

embed(w)

+

PE(pos)

=

input vector



Why sine and cosine?

Smooth, bounded functions where $PE(pos+k)$ is a linear function of $PE(pos)$ $\rightarrow$ model can easily learn relative positions. Also generalizes to longer sequences than seen in training.

Part 3

Self-Attention

The core idea that makes the whole thing work

What Problem Does Self-Attention Solve?

An example: the word 'it'

Example sentence:

*"The animal didn't cross the street because **it** was too tired."*

What does "it" refer to — the animal, or the street?

Easy for a human. Hard for an algorithm — unless it can look at all the other words.

If "it" attends to "animal" ✓

The model correctly understands the sentence.
→ Better translation, summarization, etc.

If "it" attends to "street" ✗

The model misunderstands.
→ Mistranslation, wrong meaning.

Self-attention is a mechanism that lets every word "look at" every other word, and decide which ones are relevant.

Query, Key, Value — A Library Metaphor

The three vectors that drive everything



Imagine searching for a book

You walk up with a **question** ("books about neural networks").

The library has **labels** on each shelf (titles, topics).

You compare your question to each label, then grab the **contents** of the most relevant books.



Query (Q)

What am I looking for?

comes from current word



Key (K)

What do I represent?

comes from every word



Value (V)

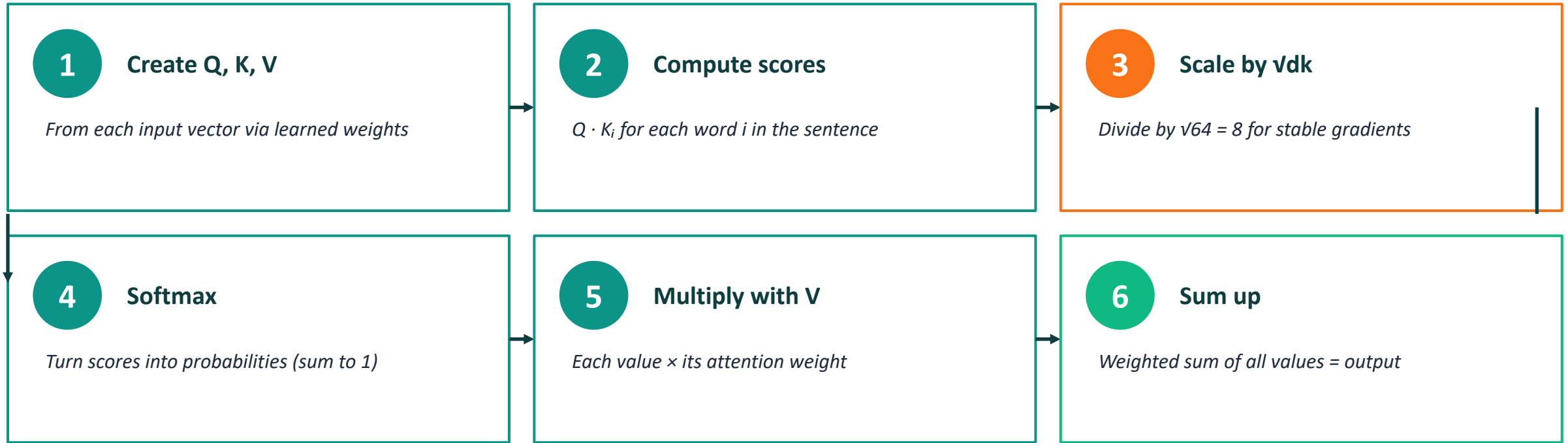
What's my content?

comes from every word

Each input vector is multiplied by 3 learned weight matrices (W^Q , W^K , W^V) to produce its Q, K, V.

Self-Attention, Step by Step

Computing attention for one word



Intuition

- Scores measure **how relevant** each other word is to me.
- Softmax ensures "relevance" sums to 100%.
- Weighted sum lets me blend in information from the most relevant words.

A Toy Example: Computing Attention for "Thinking"

Two-word sentence: "Thinking Machines"

Words and their Q/K vectors

	Query	Key	Value
Thinking	q_1	k_1	v_1
Machines	q_2	k_2	v_2

Computing attention output for "Thinking" (using q_1):

Step 2: raw scores

$$\begin{aligned} q_1 \cdot k_1 &= 112 && \text{(Thinking vs Thinking)} \\ q_1 \cdot k_2 &= 96 && \text{(Thinking vs Machines)} \end{aligned}$$

Step 3: scale by $\sqrt{d_k} = 8$

$$112/8 = 14, \quad 96/8 = 12$$

Step 4: softmax

$$[0.88, 0.12]$$

→ 88% Thinking, 12% Machines

Step 5-6: weighted sum

$$z_1 = 0.88 \cdot v_1 + 0.12 \cdot v_2$$

Softmax

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Input

Embedding

Queries

Keys

Values

Score

Divide by 8 ($\sqrt{d_k}$)

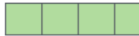
Softmax

Softmax

X
Value

Sum

Thinking

x_1 

q_1 

k_1 

v_1 

$q_1 \cdot k_1 = 112$

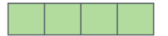
14

0.88

v_1 

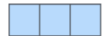
z_1 

Machines

x_2 

q_2 

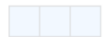
k_2 

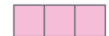
v_2 

$q_1 \cdot k_2 = 96$

12

0.12

v_2 

z_2 

Posted by Jay Alammar: The Illustrated Transformer

The Scaled Dot-Product Attention Formula

All those steps, in a single equation

Equation (1)

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V$$

Reading the equation:

$Q K^T$

compatibility scores between every query and every key (one number per pair)

$/ \sqrt{d_k}$

scaling for numerical stability (more on this next slide)

$\text{softmax}(\dots)$

turn scores into weights that sum to 1

$\dots V$

weighted sum of value vectors $\rightarrow$ the output

Why Divide by $\sqrt{d_k}$?

A subtle but important detail

⚠ The problem

Dot products grow with dimension.

If q and k have components $\sim N(0,1)$,
then $q \cdot k$ has variance d_k .

Large $d_k \rightarrow$ large dot products
 $\rightarrow$ softmax becomes spiky
 $\rightarrow$ tiny gradients
 $\rightarrow$ **no learning!**

✓ The fix

Divide by $\sqrt{d_k}$ before softmax.

Variance of $(q \cdot k) / \sqrt{d_k} = d_k / d_k = 1$.

$\rightarrow$ scores stay in a stable range
 $\rightarrow$ softmax stays well-behaved
 $\rightarrow$ gradients flow nicely
 $\rightarrow$ **training works.**

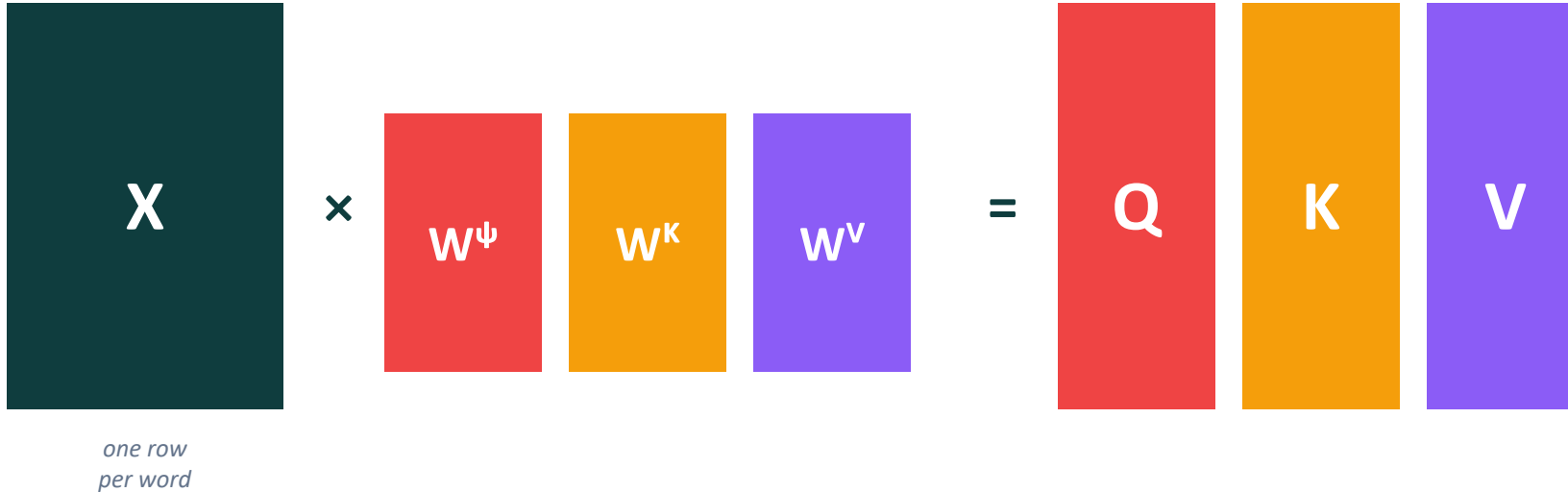
Why I'm spending a slide on this:

This kind of scaling fix is the difference between a model that works and one that doesn't. Practical deep learning is full of such details — the paper's contribution isn't just the architecture, but also figuring out what makes it train.

All at Once: The Matrix View

Why this scales — every word in parallel

Pack inputs into matrices:



Then:

$$\text{softmax}(QK^T / \sqrt{d_k}) V$$

computes attention for

ALL words simultaneously.

This is what lets the Transformer parallelize so well.

Dimensions (for reference)

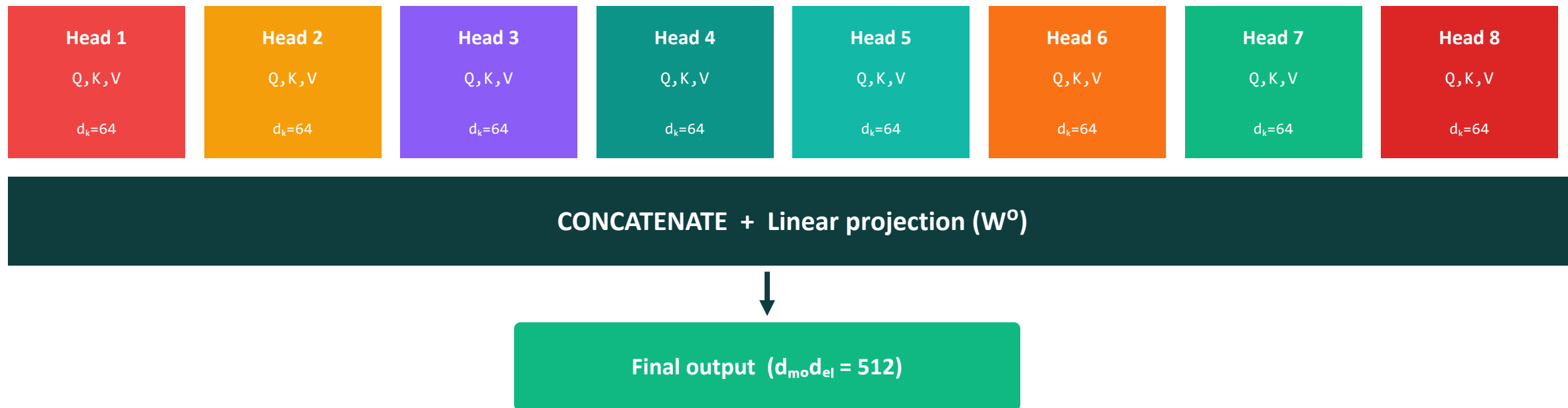
Matrix	Shape	Description
X	$n \times d_{\text{model}}$	n words, each as 512-d vector
Q, K	$n \times d_k$	$d_k = 64$ in the paper
V	$n \times d_v$	$d_v = 64$ in the paper
Output	$n \times d_v$	Same shape as V

Multi-Head Attention

Why one set of Q/K/V is not enough

Idea: do attention multiple times in parallel, with different learned projections.

h = 8 attention heads in the paper

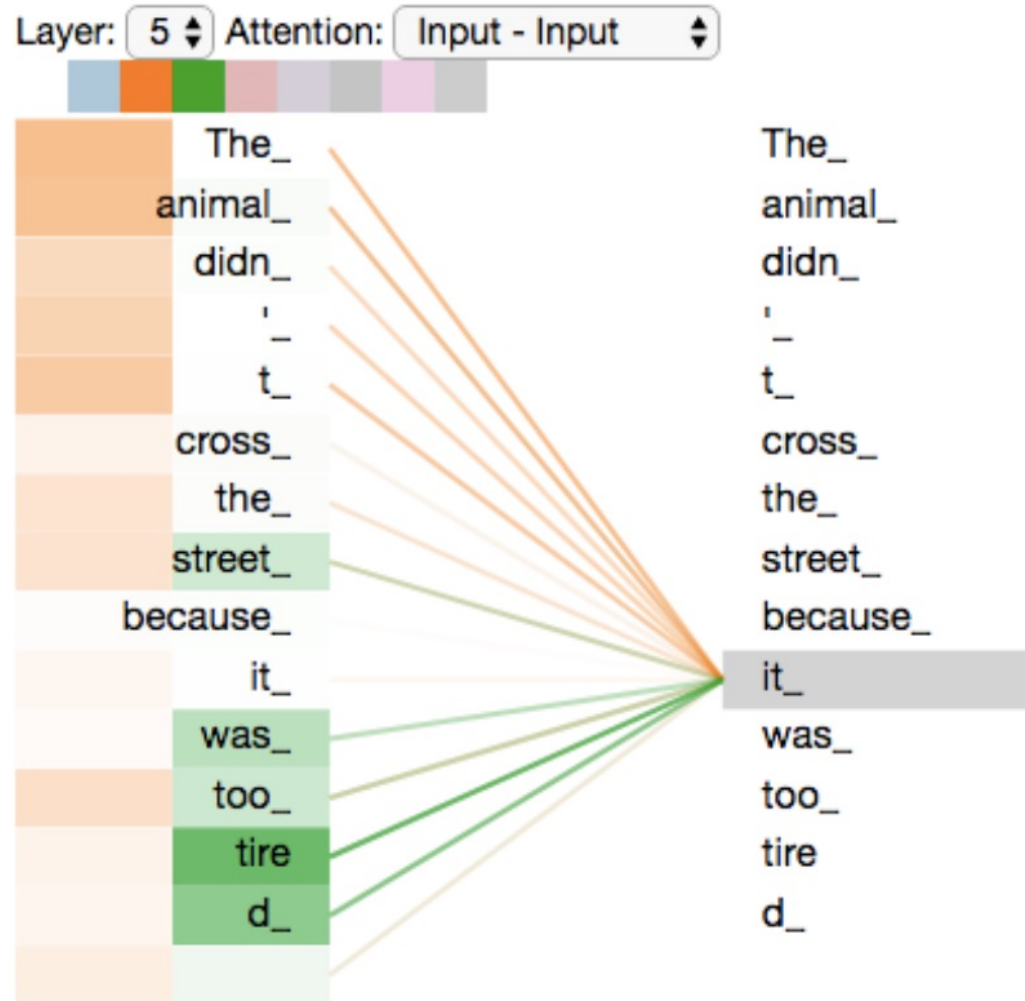


Why multiple heads?

Each head can specialize. E.g., head 1 tracks subject-verb agreement, head 2 tracks pronouns ("it" → "animal"), head 3 tracks adjectives. *One head can't do it all — averaging would wash out the specifics.*

Seeing What Attention Learns

Different heads catch different patterns



What we observe

- **Heads as specialists**
Different heads learn to track different linguistic phenomena.
- **Long-range links**
Attention easily connects words that are far apart in the sentence.
- **Pronoun resolution**
Some heads clearly attend from "it" to its antecedent (e.g., "animal").
- **Syntactic structure**
Other heads seem to track grammatical structure (subject, object, modifier).
- **Interpretability**
Unlike RNN hidden states, attention weights are visually inspectable.

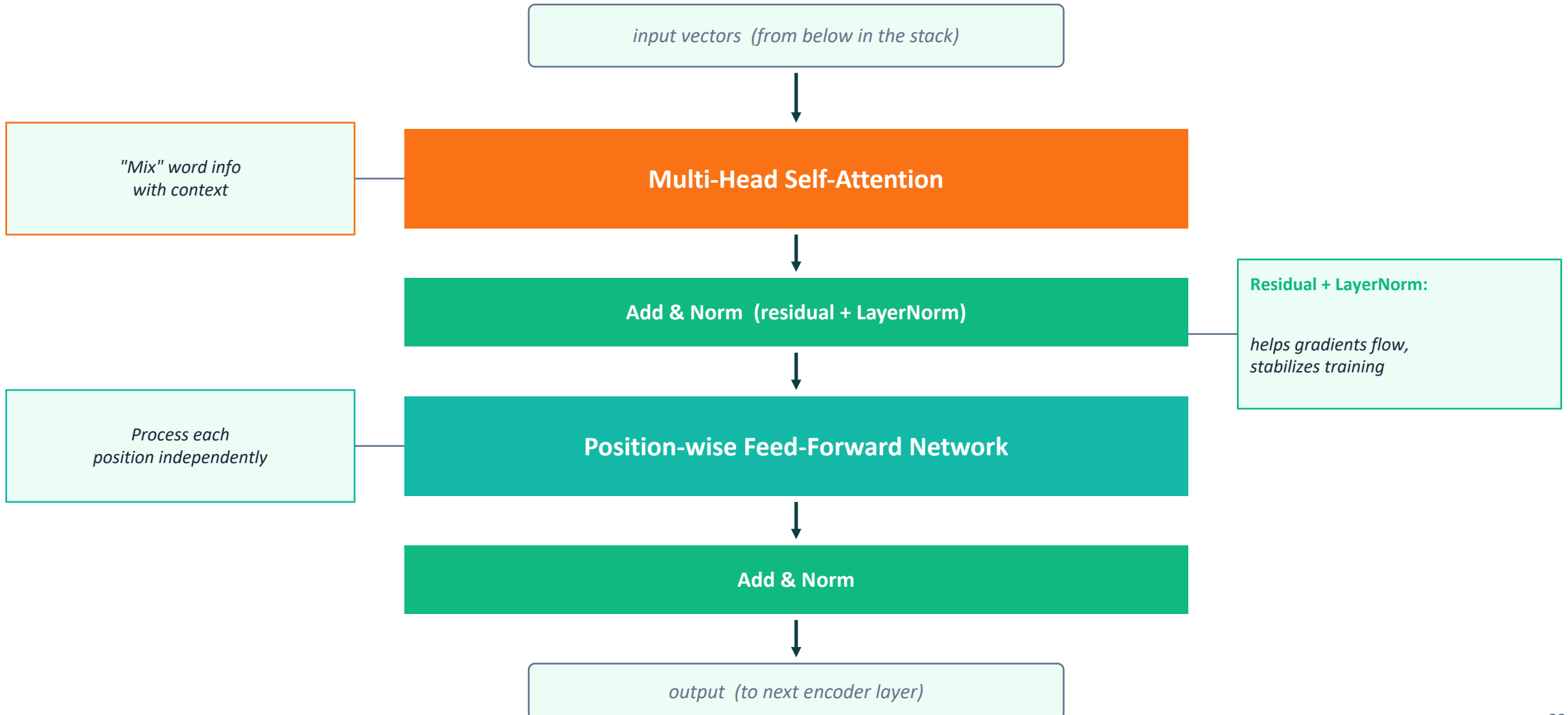
Part 4

Putting It All Together

Encoder, decoder, and the other ingredients

Inside One Encoder Layer

What happens at each of the 6 stacked layers



What's Different in the Decoder?

Two extra wrinkles

1. Masked Self-Attention

Problem:

When generating word 3, the model must not "cheat" by looking at words 4, 5, 6 in the target sentence.

Solution:

Before softmax, set attention scores for future positions to $-\infty$.
After softmax $\rightarrow$ 0 weight on future words.

Effect:

Position i can only attend to positions 1, ..., i .

2. Cross-Attention

Why?

Decoder needs to look at the source sentence (encoder output) to translate.

How?

Q from decoder (*current state*)

K, V from encoder (*source sentence*)

Same attention formula, different inputs.

This is how the decoder "knows" what to translate from.

Part 5

Why Self-Attention?

And the experimental results that proved it

Self-Attention vs RNN vs CNN

Table 1 from the paper

Layer Type	Complexity per Layer	Sequential Operations	Max Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$ ✓	$O(1)$ ✓
Recurrent	$O(n \cdot d^2)$	$O(n)$ ✗	$O(n)$ ✗
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k n)$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

Parallelization

Self-attention: $O(1)$ sequential ops.
RNN: must wait for previous step.

Long-range deps

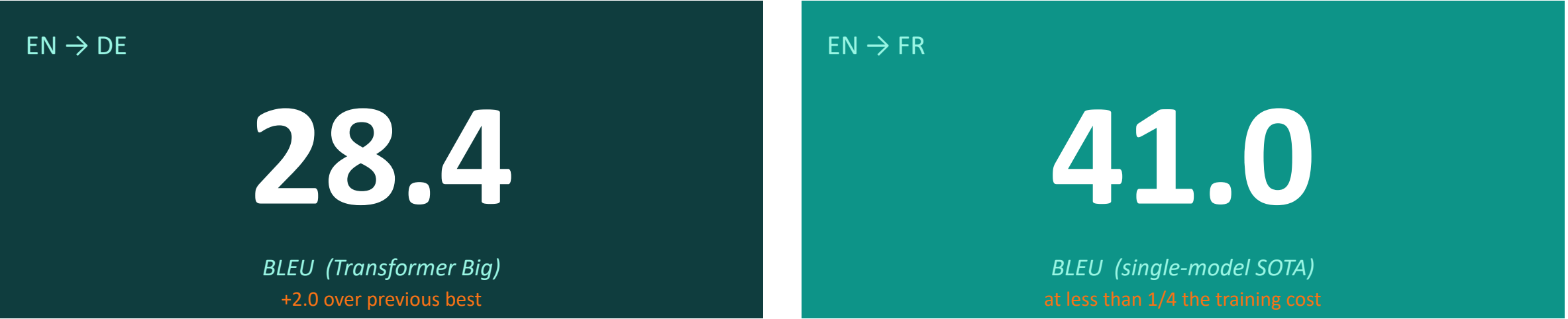
Any two positions are 1 hop apart.
In RNN: n hops to cross sentence.

Interpretability

Attention weights are inspectable.
RNN hidden states are opaque.

Results: Translation Quality

Table 2 — BLEU scores on WMT 2014



And it was much cheaper to train

Model	EN-DE BLEU	Training Cost (FLOPs)
GNMT + RL	24.6	2.3×10^{19}
ConvS2S	25.16	9.6×10^{18}
GNMT + RL Ensemble	26.30	1.8×10^{20}
Transformer (base)	27.3	3.3×10^{18} ✓
Transformer (big)	28.4	2.3×10^{19}

Putting It All Together

The full story in one slide

1

Tokenize & embed

Words → 512-d vectors

2

Add positional encoding

Inject order info

3

Encoder ×6

Self-attention + FFN, with residuals

4

Decoder ×6

Masked self-attn + cross-attn + FFN

5

Linear + Softmax

Output distribution over vocabulary

6

Output one word

Pick highest, feed back, repeat

Key insight: Replacing recurrence with attention gives parallelization + shortcuts between any positions.

Key Takeaways

Five things to remember

1

Attention replaces recurrence entirely

No RNN, no CNN — just attention + feed-forward layers.

2

Q/K/V is a soft lookup

Query says what I want, key advertises what I have, value carries content. Softmax decides who matters.

3

Scaling matters

$\sqrt{d_k}$ scaling keeps softmax in a healthy regime — essential for training.

4

Multi-head = multiple perspectives

Each head can specialize in a different linguistic relationship.

5

Massive impact

BERT, GPT, T5, ViT, AlphaFold — almost every major NN model since 2018 is a Transformer.

What Came Next (2017 → today)

The Transformer reshaped AI

2018

BERT, GPT-1

Encoder-only & decoder-only Transformers for language understanding & generation.

2019

GPT-2, T5, XLNet

Bigger models, transfer learning becomes standard.

2020

GPT-3, ViT

Few-shot learning emerges. Vision Transformer brings attention to images.

2022-23

ChatGPT, GPT-4, LLaMA

Public revolution. Transformers everywhere.

2024

Multimodal everything

Audio, video, code, biology — same architecture, different data.

All of this from a 2017 paper about machine translation. The title was right — attention really was all you needed.

Thank You

Questions & Discussion

Vaswani et al. (2017) — Attention Is All You Need
Neural Information Processing Systems (NIPS)